

Classificação de Reviews da Steam

Machine Learning Semissupervisionado com Grafo Tripartido

Estruturas de Dados 2 — UnB

João Marcelo • Lucas de Paula • Pedro Henrique • Raissa Silva • Yasmin Dayrell

O Problema

Contexto

A Steam oferece apenas rótulos binários nas avaliações:

✓ Recomendado ✗ Não Recomendado

Mas os jogadores falam sobre temas muito mais ricos:

- Problemas de performance e FPS
- Qualidade dos gráficos
- Mecânicas de gameplay
- Qualidade da narrativa

Como classificar automaticamente esses aspectos sem rótulos manuais?

Nossa Solução

ML Semissupervisionado

Aproveitamos os poucos rótulos conhecidos para propagar informação por toda a rede.

Grafo Tripartido

Estruturamos reviews, palavras e categorias como uma rede de 3 camadas interconectadas.

Label Propagation

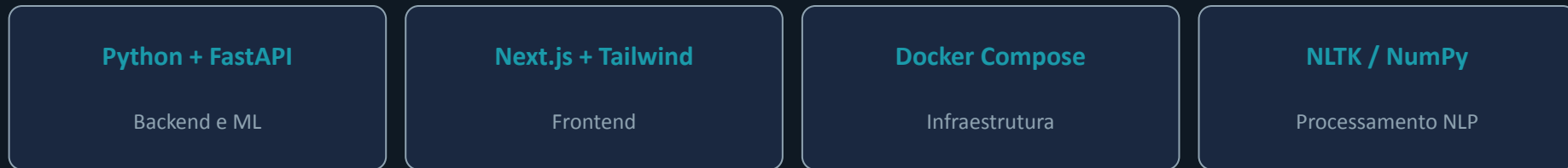
Algoritmo que propaga rótulos de categorias pelos pesos TF-IDF e PMI do grafo.

Dataset: GTA V (Kaggle / Steam API)

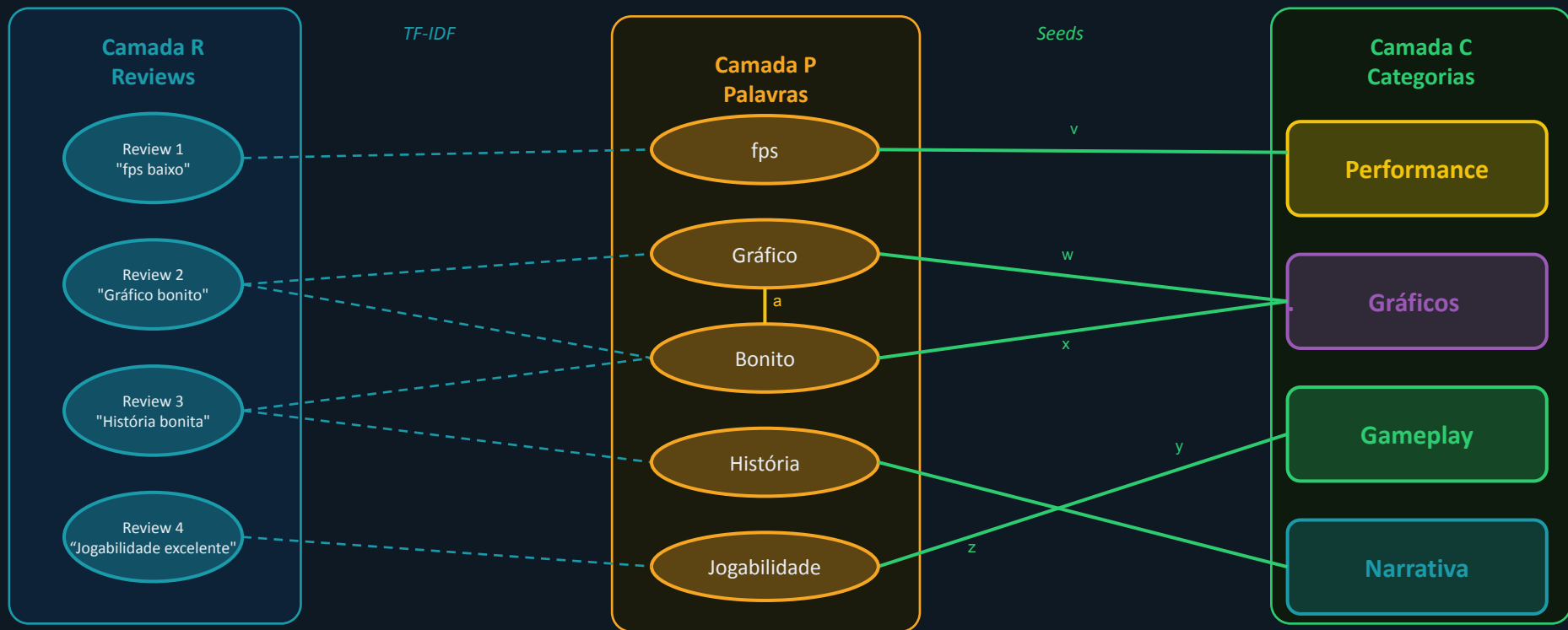
Pipeline do Sistema



Stack Tecnológico



O Grafo Tripartido



Vértices: Os Nós do Grafo

R — Nós de Review

- › Cada review do GTA V no dataset é um vértice
- › Armazena: ID único da review, texto original
- › Tipo: nó de entrada (source) da propagação
- › Quantidade: proporcional ao dataset filtrado
- › Não recebe rótulo direto — classificado pela propagação

P — Nós de Palavra

- › Tokens únicos após pré-processamento NLP
- › Aplicado: lower, tokenização, remoção stopwords, stemming
- › Nós intermediários: conectam reviews a categorias
- › Identificados por índice em lista ordenada
- › Mais denso: bridge entre as duas camadas

C — Nós de Categoria

- › 4 nós fixos: Performance, Gráficos, Gameplay, Narrativa
- › Nós destino (sink) da propagação de rótulos
- › Cada review converge para 1 categoria dominante
- › Inicializados com seeds: palavras-chave polarizadas
- › "fps" → Performance, "visual" → Gráficos, etc.

Arestas: As Conexões do Grafo

Aresta $R \leftrightarrow P$ — Review $\leftrightarrow$ Palavra (peso: TF-IDF)

TF-IDF (Term Frequency $\times$ Inverse Document Frequency) mede a importância de uma palavra em um documento relativo ao todo.

Fórmula: $w(r,p) = \text{tf}(p,r) \times \log(N / \text{df}(p))$ — Alta frequência local + baixa global = peso alto.

Aresta $P \leftrightarrow P$ — Palavra $\leftrightarrow$ Palavra (peso: PMI)

PMI (Pointwise Mutual Information) captura a coocorrência de pares de palavras.

Alta coocorrência acima do acaso = peso alto.

$$\text{NPMI}(x; y) = \frac{\text{PMI}(x; y)}{-\log P(x, y)}$$

Aresta $P \leftrightarrow C$ — Palavra $\leftrightarrow$ Categoria (seeds manuais)

Seeds: palavras-chave óbvias são manualmente ligadas a categorias com peso máximo (1.0).

Exemplos: "fps", "lag", "performance" $\rightarrow$ ⚡ Performance | "visual", "graphic" $\rightarrow$ 🎨 Gráficos | "story", "plot" $\rightarrow$ 📖 Narrativa

Lista Ordenada + Busca Binária

Todo mapeamento de nomes para índices é feito via **Busca Binária $O(\log N)$** em listas ordenadas.

Como funciona

1. Vocabulário é mantido como lista `sorted_words`:

```
sorted_words = ["beauti", "drop", "fps", "lag", "visual", ...]
```

2. Para buscar o índice de "fps":

```
→ bisect_left(sorted_words, "fps") → índice 2
```

3. O índice é usado para acessar as listas de adjacência

Busca por "lag" na `sorted_words`

beauti	drop	fps	lag	story	visual
id x 0	1	2	3	4	5

mid=2 (fps)

< lag → busca direita →

✓ Encontrado!

Índice = 3



Aplicação: Ao adicionar aresta (`review_0`, "fps", 0.82), o sistema usa bisect para localizar o índice de "fps" sem jamais usar um dict.

Label Propagation: O Algoritmo Central

1

Inicialização:

Seeds manuais definem scores iniciais: ex: `fps.score["Performance"] = 1.0`, todos os outros nós começam zerados.

2

Iteração:

Para cada nó R , calcula: $\text{score_r}[C] = \sum (w(r,p) \times \text{score_p}[C])$ para todos os vizinhos P . Para nós P , propaga via PMI para outros P .

3

Normalização:

Os scores de cada review são normalizados para somar 1.0, permitindo comparação entre diferentes comprimentos de texto.

4

Convergência:

O processo repete até que a diferença máxima de score entre iterações seja menor que ϵ (epsilon) — tipicamente 0.001.

5

Classificação:

Categoria final = $\text{argmax}(\text{score_r}[C])$. Ex: se Performance=0.91, Gráficos=0.06, Gameplay=0.02, Narrativa=0.01 → ⚡ Performance.

Convergência Numérica do Algoritmo

O que é max_delta?

A cada iteração, calcula-se a maior variação absoluta entre os scores antigos e novos de todos os nós e categorias.

Fórmula

$$\text{max_delta} = \max(|\text{score}_{\text{novo}} - \text{score}_{\text{antigo}}|)$$

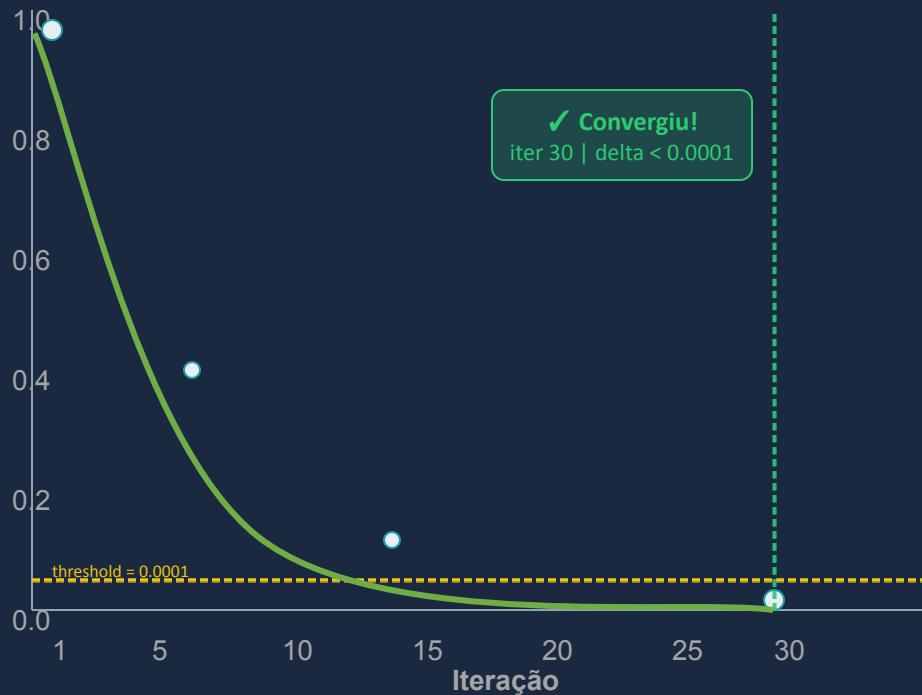
Critério de Parada

O algoritmo converge quando:

$$\text{max_delta} < \text{threshold} \rightarrow \text{threshold} = 0.0001$$

Notebook: threshold = 0.0001 | Iterações: até 50

Gráfico: max_delta por iteração



BFS: Rastreabilidade de Categorias

Após a review ser classificada, o BFS reconstrói o caminho mais curto entre o nó da Review e o nó da Categoria, revelando a **razão lógica** da decisão.

Como funciona

1. Imagine uma review classificada como "**Gráfico**".
2. Rodamos o BFS partindo do nó *Review* até o nó da categoria destino.
3. O resultado é o **Caminho Mais Curto** no grafo tripartido (R-P-C).
4. Isso transforma a "caixa preta" da IA em um processo rastreável.

Exemplo de Output do BFS

```
Review_142
  ↓ (contém)
  "lindo"
  ↓ (coocorrência PMI)
  "visual"
  ↓ (pertence a)
  "Gráfico"
```



Aplicação: O BFS utiliza a lista de adjacência para navegar entre os vértices R, P e C de forma eficiente.

Conclusão



Grafo Tripartido (R-P-C) como estrutura central para modelar relações texto $\leftrightarrow$ palavra $\leftrightarrow$ categoria



TF-IDF (R $\leftrightarrow$ P) e PMI (P $\leftrightarrow$ P) como pesos matematicamente fundamentados para as arestas do grafo



Listas de Adjacência implementadas do zero — sem networkx, respeitando as restrições da disciplina



Seeds P $\leftrightarrow$ C permitem semissupervisionamento: rótulos se propagam sem precisar anotar todos os dados



Busca Binária $O(\log N)$ substitui hash tables na indexação de vértices, provando domínio das EDs básicas



Arquitetura full-stack: FastAPI + Next.js + Docker — reproduzível e visualizável no frontend